

Basket Management Differences with PWA Kit

PWA Kit and Storefront Next rely on the Shopper Baskets API, but they differ in where basket state is managed and how it is hydrated in the app lifecycle. PWA Kit commonly uses hook-driven basket reads and mutations in components, while Storefront Next uses middleware, request context, and route loaders and actions to manage basket state with a server-first model.

Contents

Architecture

[Request and State Lifecycle](#)[Migration Considerations](#)[Key Similarities](#)[References](#)

Architecture

Aspect	PWA Kit	Storefront Next
Basket API baseline	Shopper Baskets V1 is available and remains the default for many existing imports	Basket flows are implemented with <code>shopperBasket</code> patterns in loaders/actions
Primary integration model	<code>@salesforce/commerce-sdk-react</code> hooks in components	Middleware + request context route loaders/actions
Basket state location	React Query cache and component hook state	Cookie snapshot the server plus optional client hydration in props
Hydration strategy	Data fetched when basket hooks run in UI	Lazy by default hydrate full basket only when requested

Request and State Lifecycle

Phase	PWA Kit	Storefront Next
Initial request	Basket data is typically fetched through basket	Middleware reads basket cookie snapshot and



Basket creation/access	Hook and mutation flows resolve or create basket during shopper actions	<code>ensureBasketId</code> and server helpers manage basket identity in loaders/actions
Client updates	Mutations update React Query caches and UI state	Actions update server resource context; client can sync through provider updater hooks
Post-checkout cleanup	App-specific hook/mutation flows clear basket	<code>destroyBasket()</code> and provider reset hooks clear server/client basket state

Migration Considerations

When moving basket functionality from PWA Kit to Storefront Next:

1. Move basket reads out of components and into route loaders when possible.
2. Move basket mutations to route actions and update request-scoped basket context.
3. Treat basket cookie snapshot data as the default source for lightweight UI state (for example, cart badge counts).
4. Use lazy hydration for full basket payloads only in routes or components that require full basket detail.
5. Keep basket API usage consistent across the codebase to avoid stale or mismatched basket state behavior.

Key Similarities

- Both storefronts use SCAPI basket endpoints for cart operations.
- Both support guest and authenticated shopper basket flows.
- Both can preserve basket continuity across shopper navigation.

References

- [Shopper Baskets V1 API Reference](#)
- `sfnext-basket-management`



Try for free



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUNITY

Trailblazer Community
Events and Calendar
Partner Community
Blog
Salesforce Admins
Salesforce Architects

© Copyright 2026 Salesforce, Inc. [All rights reserved.](#) Various trademarks held by their respective owners. Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#) [Use of Cookies](#)

[Cookie Preferences](#)  [Your Privacy Choices](#)